TD 2 – Bases de Python (cont.)

Intelligence Artificielle Hybride et Apprentissage Automatique

Exercice 1 - PARCOURIR UN MOT

Sollicitez l'utilisateur d'entrer un mot au clavier, puis :

1. Afficher chaque lettre du mot sur une ligne.
2. Compter le nombre de lettres du mot (sans utiliser `len`).
3. Afficher les lettres en majuscules.

Exercice 2 - PARCOURIR UN TEXTE

Écrivez un programme Python qui étant donné le texte suivant « ***Python est TOP à 100% !!! J'aime coder en 2026 :)*** » , parcourt ce texte **caractère par caractère** à l'aide d'une boucle `for` et en utilisant l'instruction `match/case`, compte le nombre de :

1. lettres majuscules,
2. lettres minuscules,
3. chiffres,
4. espaces,
5. caractères spéciaux (tout caractère qui n'est ni une lettre ni un chiffre ni un espace).

Finalement, affichez le nombre de caractères de chaque catégorie.

Exercice 3 - FONCTION ANALYSER_TEXTE

Modifiez le code de l'exercice précédent pour :

1. Définir une fonction `analyser_texte(text)` qui reçoit un texte en **paramètre**, et compte la quantité de chaque type de caractère (exercice n° 2)
2. Demander à l'utilisateur de saisir un texte au clavier.
3. Appeler la fonction `analyser_texte` avec le texte saisi en paramètre.
4. Afficher les résultats.

Exercice 4 - MESSAGE CACHE

Nous souhaitons créer un petit encodeur de texte. Étant donné un dictionnaire dont les clés sont des lettres (en minuscules) et les valeurs sont des symboles ou des lettres d'autres alphabets (grec, arabe, hébreu, etc.), implémentez une fonction `message_cache(texte, code)` qui :

1. Reçoit en paramètres un `texte` et un dictionnaire `code`,
2. Remplace chaque caractère du `texte` qui est présent comme clé dans le dictionnaire par sa valeur « secrète »,
3. Laisse inchangés les caractères qui ne sont pas dans le dictionnaire.

La fonction retourne le texte transformé (c'est-à-dire le message « caché »). Testez ensuite la fonction sur une phrase de votre choix.

Exercice 5 - PORTEE DES VARIABLES

Considérez le programme suivant :

```
x = 10
print("A) x =", x)

def calculer(x):
    print("B) Dans la fonction (avant modification), x =", x)
    x = x + 5
    print("C) Dans la fonction (après modification), x =", x)

x = 3
print("D) Avant l'appel de la fonction, x =", x)

calculer(x)

print("E) Après l'appel de la fonction, x =", x)

x = x + 2
print("F) x =", x)
```

Indiquez l'ordre d'exécution (quelle ligne s'exécute en première, laquelle en deuxième, etc.) et la valeur imprimée de X pour chaque étape :

Ordre d'exécution	**Ligne de code**	**Valeur de X**
	A) x =	
	B) Dans la fonction (avant modification), x =	
	C) Dans la fonction (après modification), x =	
	D) Avant l'appel de la fonction, x =	
	E) Après l'appel de la fonction, x =	
	F) x =	

Exercice 6 - ANALYSE DES PHRASES D'UN TEXTE

Écrire un programme Python qui :

1. Demande à l'utilisateur de saisir un texte comportant plusieurs phrases.
2. Découpe le texte en phrases (séparées par des points).
3. Affiche chaque phrase sur une ligne.
4. Affiche le nombre total de phrases.
5. Pour chaque phrase, affiche le nombre de mots et de caractères qu'elle contient.

Exercice 7 - NETTOYAGE D'UN CATALOGUE D'ŒUVRES

On dispose d'un catalogue d'œuvres littéraires représenté par une liste de tuples de la forme : (id, titre, auteur, année). Certaines années peuvent être mal saisies (par un mauvais type ou une valeur incohérente).

```
catalogue = [("D01", "Du côté de chez Swann",  "Proust",      1913),
             ("D02", "L'Étranger",             "Camus",       "Camus"),
             ("D03", "La Nausée",              "Sartre",      1938),
             ("D04", "Les Fleurs du mal",      "Baudelaire",  " 1857"),
             ("D05", "Le Joueur d'échecs",     "Zweig",       2942)]
```

1. Fonction *normaliser_annee(catalogue)*

Écrire une fonction normaliser_annee qui parcourt le catalogue et renvoie un **nouveau catalogue normalisé**. Pour chaque entrée :

- si l'année est un entier, elle est conservée ;
- si l'année est une chaîne de caractères :
  - si elle représente un nombre, elle est convertie en entier et le message suivant est affiché : ***L'année de DXX a été mise sous forme de int***
  - sinon, l'entrée est ignorée et le message suivant est affiché : ***L'année de DXX est une chaîne de caractères non numérique***
- dans tous les autres cas, l'entrée est ignorée et le message suivant est affiché : ***L'année de DXX a un type invalide***

2. Fonction `valider_annee(catalogue)`

Écrire une fonction valider_annee qui prend en entrée un catalogue déjà normalisé et retourne un **nouveau catalogue valide**. Pour chaque entrée :

- Vérifier que l'année est **inférieure ou égale à l'année en cours**. Pour obtenir l'année courante, il vous faudra ajouter l'import suivant :

  ```
  from datetime import date
  ```

- Si l'année est future, l'entrée est ignorée et le message suivant est affiché : ***L'année de DXX dépasse la date actuelle***
- Sinon, l'entrée est conservée.